Introduction to Operating Systems

Lecture 1 — Functions, Evolution, Structure, Services, and System Calls

Dr. Auqib Hamid Lone
Assistant Professor in Computer Science & Engineering

PCC CS-502: Operating Systems

Department of Computer Science & Engineering
Government College of Engineering & Technology (GCET), Ganderbal

October 29, 2026

Content Not Click

Welcome: What Sits Between the Program and the Machine?

The big question

You run `cp notes.txt backup.txt`. Between your keystroke and the spinning disk are the **shell**, the **kernel**, a **system-call** boundary, a **device driver**, and a **controller**. Which of these is the operating system — and what exactly does it do?

- A computer is *hardware*: CPU, memory, disks, devices.
- A program is *logic*: code that wants to use that hardware.
- The **operating system (OS)** is the software layer that turns hardware into a safe, shared, usable machine.

Running thread for today

We trace one real command — `cp notes.txt backup.txt` — from the shell all the way to the disk controller, and back.

Roadmap: Five Questions, One Command

1. **What is an OS?** — resource manager, control program, extended machine.
2. **How did we get here?** — batch, multiprogramming, time-sharing, real-time.
3. **What protects the machine?** — dual-mode operation.
4. **How is it built?** — simple, layered, microkernel, modular, virtual-machine structures.
5. **How do programs talk to it?** — OS services, system calls, and system programs.

Concepts That Click

Before we can say what an OS does, we need a working definition — one that survives every machine from a thermostat to a datacenter.

Three Ways to Say the Same Thing

Definition 1 — the resource manager

A program that manages the computer's resources: the CPU, memory, storage, and I/O devices. It decides *who* gets *what*, and *when*.

Definition 2 — the control program

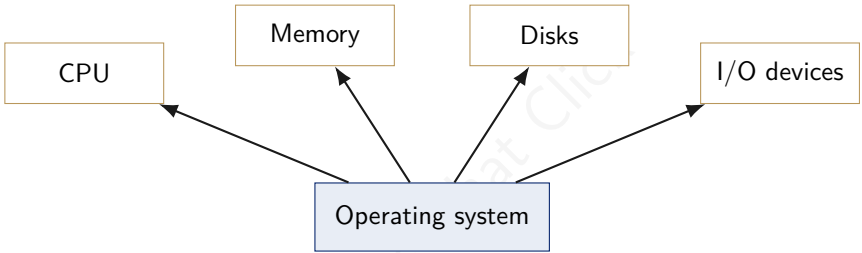
A program that controls the execution of user programs to prevent errors and improper use of the computer — the referee, not just the allocator.

Definition 3 — the extended machine

A program that hides the messy hardware behind clean abstractions (files, processes, sockets) so programmers never touch a register directly.

All three describe the same software; which one you emphasize depends on what the machine is for (Silberschatz et al., Ch. 1).

The OS as Resource Manager



- For cp: the CPU runs its instructions, memory holds its buffer, the disk stores the file, the I/O path moves the bytes.
- The OS **multiplexes** scarce resources across competing programs and resolves conflicting requests.

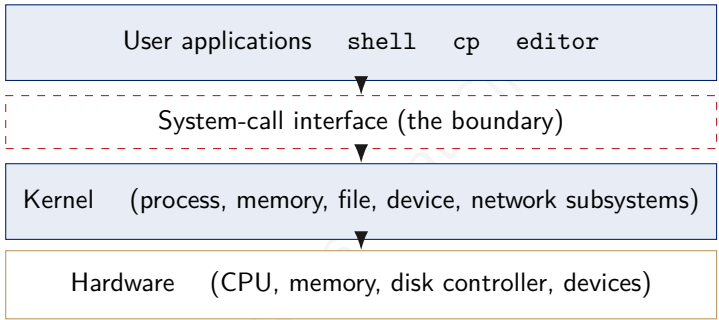
The OS as Extended Machine (Abstraction)

- A raw disk is an array of blocks addressed by (cylinder, head, sector). A programmer should not have to know that.
- The OS offers a **file**: a named stream of bytes. Open, read, write, close — hardware details gone.
- Likewise: a raw CPU becomes a **process**; a network card becomes a **socket**; physical frames become **virtual memory**.

Why abstraction is the point

The same program must run on different disks, CPUs, and networks. The OS *decouples* the program from the hardware — this is what makes portability possible.

Where the OS Sits: The User/Kernel Boundary



- **User mode**: restricted; privileged instructions are illegal.
- **Kernel mode**: unrestricted; the OS runs here.

Socratic Check 1

Think, then answer

Is a web browser an operating system? Is a device driver? Is firmware?

- **Browser** — no: it is an *application* that *uses* OS services (files, sockets, processes).
- **Device driver** — in the kernel: it is OS code, one layer down.
- **Firmware** — between: it boots and initializes hardware before the OS runs, but is not the OS.

Concepts That

The OS did not appear fully formed. Each feature exists because an earlier design failed under a real workload.

Concepts Click

Evolution I: Serial Processing and Batch Systems

- **Serial processing** (1940s–50s): the operator loads one job, the program runs to completion, then the next. No OS — a human is the scheduler.
- The problem: the CPU idles while the operator mounts tapes and cards. Utilization is terrible.
- **Batch operating system**: jobs of similar type are grouped into a batch and run by a **resident monitor** — a small program that transfers control to the next job automatically.
- A **job-control language (JCL)** tells the monitor what to run.

The lesson that never goes away

Any time the CPU waits on a slow human or device, an OS should take over the waiting. This is the seed of multiprogramming.

Evolution and its design pressures: Silberschatz et al., Ch. 1; Stallings, Ch. 2 [2].

Evolution II: Multiprogramming and Spooling

- **Multiprogramming**: keep several jobs in memory at once; when the running job waits for I/O, switch to another. The CPU stays busy.
- **Spooling** (Simultaneous Peripheral Operation On-Line): buffer input/output on disk so slow devices run in parallel with the CPU — the printer never stops the computer.
- **Time-sharing (multitasking)**: extend multiprogramming so the CPU switches so fast that every user has an *interactive* session, as if the machine were theirs alone.

Multiprogramming maximizes CPU utilization; time-sharing minimizes response time. The two goals can conflict.

Concepts Tinubu

Evolution III: Interactive and Real-Time Systems

- **Interactive / time-sharing** systems (CTSS, Multics, UNIX): the user gets a shell and a prompt; the OS maintains the illusion of private ownership.
- **Real-time systems**: correctness depends on *when* the result arrives, not just on the values.
  - **Hard real-time** — a missed deadline is a failure (fly-by-wire, pacemakers, ABS brakes).
  - **Soft real-time** — a missed deadline degrades quality (video streaming, online games).

Two axes, not one

Design pressure comes from *utilization* (batch), *responsiveness* (time-sharing), and *predictability* (real time). Modern general-purpose OSes (Linux, Windows, macOS) try to satisfy all three at once.

System Protection: Why the Boundary Must Be Enforced

- If a user program could execute privileged instructions, one buggy program could overwrite the OS or another process.
- **Dual-mode operation**: a hardware **mode bit** selects user (1) or kernel (0) mode. Privileged instructions are legal only in kernel mode.
- **System call**: the *only* legal doorway from user mode to kernel mode — a controlled trap to the kernel.
- **Memory protection**: base/limit registers (or page tables) ensure a process touches only its own memory.
- **Timer interrupt**: guarantees the kernel regains control — a runaway process cannot monopolize the CPU forever.

Protection is not a feature bolted on top; it is a hardware/OS contract that everything else depends on (Stallings, Ch. 3 [2]).

If the OS is the trusted layer, how do we build such a large, long-lived program without it collapsing under its own complexity?

Concepts Click

Five Ways to Organize a Kernel

- **Simple / monolithic** — one big program, no clean modules (early MS-DOS; UNIX is mostly monolithic).
- **Layered** — the OS is a hierarchy of levels; each level uses only the one below (MULTICS, THE).
- **Microkernel** — a tiny kernel (IPC, scheduling, basic memory) plus user-space servers (Mach, QNX, L4).
- **Modular** — a core kernel with loadable modules (Solaris, Linux, macOS) — dynamically extensible.
- **Virtual machines** — the OS *itself* treats hardware virtually (VMware, Xen, KVM).

The trade-off across all five: simplicity and safety vs. performance and flexibility.

Concepts That Click

Monolithic vs. Layered

Monolithic

All services in one address space; any part can call any other. Fast: direct procedure calls. Fragile: no protection between subsystems, hard to debug. *MS-DOS* had no separation at all.

Layered

Level N may use only level $N-1$. Easier to design and debug, layer by layer. Hard to *define* the layers, and each boundary adds call overhead. *THE* OS used six layers.

The design question: how much structure can you add before the overhead outweighs the benefit?

Microkernel vs. Modular vs. Virtual Machine

- **Microkernel**: push everything but the minimum into user space. Small, reliable, secure, portable — a crash in a file server does not kill the kernel. Message-passing overhead; performance can suffer.
- **Modular**: the kernel is a set of *loadable* modules with defined interfaces. Best of both — direct calls for the core, dynamic loading for drivers and file systems. This is *Linux* today.
- **Virtual machine**: a hypervisor presents many virtual copies of the hardware; each guest runs its own OS. Perfect isolation, consolidation. Complex, and I/O virtualization is expensive.

Read the syllabus's Stallings Ch. 1–2 for the historical thread that produced these designs.

Structure is for the builders. Users and programmers only ever see one thing: the services the OS exposes, and the calls that reach them.

Concept Clinic

OS Services: Two Audiences

Services for the user

- **Program execution** — start, run, and stop programs.
- **I/O operations** and **file-system manipulation** (create, read, write, delete, list).
- **Communication** — between processes on one machine or across a network (shared memory, message passing).
- **Error detection** — catch CPU, memory, I/O, and program errors.

Services for system efficiency

- **Resource allocation** among concurrent users.
- **Accounting** — which user used how much.
- **Protection and security** — control who accesses what.

System Calls: The Program's Doorway to the Kernel

- A **system call** is a function the OS provides to programs. It *traps* from user mode into kernel mode — privileged work happens safely.
- System calls come in families:
  - **Process control** — `fork`, `exec`, `wait`, `exit`.
  - **File management** — `open`, `read`, `write`, `close`, `unlink`.
  - **Device management**, **information maintenance**, **communication**, **protection**.
- Most APIs (POSIX, Win32) *wrap* system calls; a single library call may make several.

The API is what the programmer sees; the system call is what the kernel actually executes.

Concepts, Not Code

Worked Thread: `cp notes.txt backup.txt`, Front to Back

1. `cp` calls `open("notes.txt", O_RDONLY)` — a **system call**: mode switch, path resolution, permission check.
2. A file descriptor returns; the kernel creates a **file control block** entry recording the file's location.
3. `read`/`write` move bytes through a user buffer, each a separate trap; the kernel hands block requests to a **device driver** that drives the disk **controller**.
4. `close` releases the descriptor; success is silent.

The point of the thread

Four user-visible system calls, one mode switch each — but an enormous amount of OS machinery runs *behind* them (generic: Silberschatz et al., Ch. 2; concrete kernel: Bovet & Cesati, Ch. 10).

Passing Parameters, and System Programs

Three ways to pass parameters to a system call

- **Registers** — simplest; limited by register count.
- **Block / table** — the address of a parameter block is passed in a register (Linux, Solaris).
- **Stack** — the program pushes parameters and the OS pops them.

System programs

User-level tools built *on top of* system calls: shells, compilers, file managers, editors, and the utilities in `/bin`. They are not part of the kernel, but they are part of the "OS" a user experiences.

Takeaways

- An OS is the **resource manager**, the **control program**, and the **extended machine** — three views of one layer.
- Its history is a response to **utilization** (batch → multiprogramming), **responsiveness** (time-sharing), and **predictability** (real time).
- **Dual-mode operation** plus a **timer** and **memory protection** make the boundary enforceable.
- Structure is a spectrum: **monolithic** → **layered** → **microkernel** / **modular** / **virtual machine**.
- Programs reach the OS through **system calls**; system programs are convenience layers built on them.

Concepts That Click

Socratic Check 2

Think, then answer

A runaway user process enters an infinite loop with no I/O. Why does it not hang the machine forever? Name the exact mechanism.

- The **timer interrupt**: the kernel programs a hardware timer to fire after a quantum; on the interrupt, control returns to the kernel, which can preempt the process.
- Without a timer, dual-mode alone would not be enough — a process could stay in user mode forever and never trap out.

Concept Thank

Socratic Check 3

Think, then answer

Is the shell part of the operating system? Where does the answer change the design?

- The shell is a **system program** / command interpreter — a user program that issues system calls, not kernel code.
- It is anyway the *user-visible* face of the OS; that is why UNIX's structure (kernel + many small user-space tools) is so influential — and why the interface can change without touching the kernel.

Next: OS Operations and System Protection

- Today: what the OS *is* and what it *exposes*.
- Next lecture: how the OS *stays in control* — dual-mode operation, privileged instructions, and the timer that makes preemption possible.

Reading

Silberschatz et al., *Operating System Concepts*, Ch. 1 §1.5 [1]; Stallings, *Operating Systems: Internals and Design Principles*, Ch. 3 §3.5–3.6 [2].

Code That Click

Further Reading (Page-Cited)

- A. Silberschatz, P. B. Galvin, G. Gagne, *Operating System Concepts*, 8e — Ch. 1 (pp. 3–47), Ch. 2 §2.1–2.8 (pp. 49–89).
- W. Stallings, *Operating Systems: Internals and Design Principles*, 6e — Ch. 1 (§1.4 interrupts, §1.7 I/O techniques), Ch. 2 (§2.2 OS evolution, pp. 55–64), Ch. 3 (pp. 107–152).
- D. P. Bovet & M. Cesati, *Understanding the Linux Kernel*, 3e — Ch. 1 (basic concepts, files/inodes), Ch. 10 (system calls).
- M. J. Bach, *The Design of the UNIX Operating System*, 1986 — the classic UNIX counterpart.

Page-verified via each book's `index.md` under `master_supporting_docs/CS502/supporting_books/`.

Concepts Tim Cook

References I


Abraham Silberschatz, Peter B. Galvin, and Greg Gagne.
Operating System Concepts.
John Wiley and Sons, 2018.


William Stallings.
Operating Systems: Internals and Design Principles.
Prentice Hall, 2001.

Concepts That Click